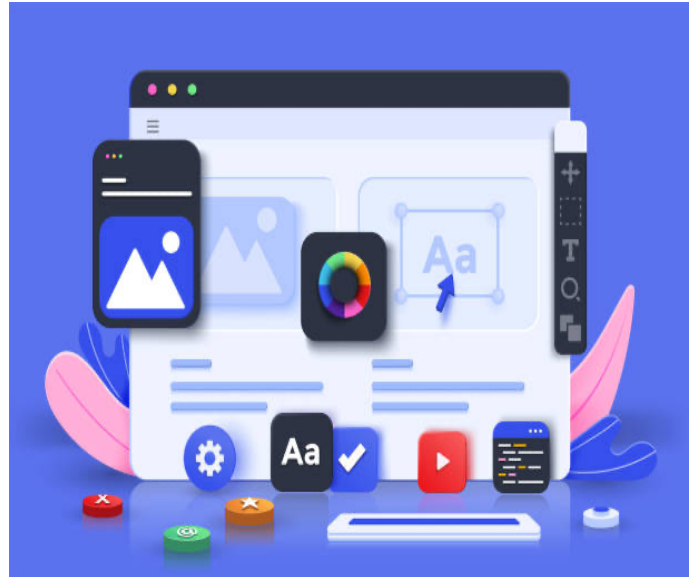


SD

—



Module Mijn eerste webgame

Instructie 08 – Refactoring

- Leesbaarder maken van de code met constanten
- Helperfunction maken om de IF-statement leesbaarder te maken

Auteur:	Johan Strootman
Afkorting:	SMN
Versie:	juli 2023

Inhoudsopgave

Inhoudsopgave	3
INLEIDING	5
VOORBEELD(EN)	6
Onleesbaarheid van de code	6
Slecht te onderhouden code	7
Wat gaan we doen?	7
REFACTORING SPELERNUMMERS	8
Inleiding	8
Hoe pakken we dit aan?	8
Waarom een constante variabele?	8
Hoe ziet dat er dan uit?	9
Hoe ziet dan de rest van onze code eruit?	9
VOORBEELD 1: VOOR DE WIJZIGING	9
VOORBEELD 1: NA DE WIJZIGING	9
VOORBEELD 2: VOOR DE WIJZIGING	10
VOORBEELD 2: NA DE WIJZIGING	10
REFACTORING HARDCODED IMAGES	11
Inleiding	11
Eén constante voor alle afbeeldingen	11
Afbeeldingsbestanden	11
Namen van afbeeldingsbestanden in een variabele/constante	12
De constante	12
img/empty.jpg	13
Oeps	13

De oplossing	13
REFACTORING VAN CEL NUMMERS	14
Inleiding.....	14
Hoe ziet de code er dan uit?	15
Hoe ziet code er dan uit als we de constanten gaan gebruiken?	15
VOOR DE WIJZIGING.....	15
NA DE WIJZIGING.....	15
REFACTORING IF-STATEMENT IN CELLCLICK FUNCTIE	16
Inleiding.....	16
Zoals het nu is.....	17
Helperfunction: Brainstormen.....	18
Hoe maken we hier een eigen functie van?	18
Wat we niet moeten doen	18
Wat we wel moeten doen.....	19
Codering van de helperfunction	20
Toelichting.....	20
Hoe gaan we de controle dan aanpassen?	21
CEL NUMMERS	21
VOOR DE AANPASSING	21
NA DE AANPASSING	21
SPELERS AFBEELDING.....	22
De functie een WAAR of ONWAAR terug laten geven	22
Wat returnen we dan?	22
De complete helperfunction	23
DE NIEUWE HELPERFUNCTION GEBRUIKEN	24
Inleiding.....	24
De nieuwe IF-statement.....	25

INLEIDING

Wat is refactoring en waarom passen we dit toe?

Het komt voor dat je tijdens het coderen niet altijd gelijk ziet dat bepaalde stukken code o.a. efficiënter kan. Het werkt en het lijkt goed zo. Maar achteraf gekeken zul je in je eigen code dingen zien waarvan je dan beseft 'Hé, maar dit kan beter'.

De aanpassingen die je dan aanbrengt in je code is dus refactoring.

Refactoring passen we ook toe om code beter onderhoudbaar te maken. En soms is refactoring ook alleen maar juiste documentatie in de code te plaatsen. Maar soms is het ook hele stukken code verschuiven naar een b.v. een eigen functie, een functie met een duidelijke naam die aangeeft wat de code doet.

VOORBEELD(EN)

Onleesbaarheid van de code

Een simpel voorbeeld van onleesbaarheid in onze code is het feit dat we nu op verschillende plaatsen de namen van images (symbolen voor spelers b.v.) **hardcoded** hebben staan en daarmee niet echt duidelijk maken welke image bij welke speler hoort.

Hieronder zie je 3 voorbeelden uit onze eigen code:

```
element_turn_image.src = (current_player == 1 ? 'img/circle200x200.png' : 'img/cross200x200.png');
```

Figuur 1 - Voorbeeld 1 van hardcoded images

```
if (event_element.target.src.includes('img/empty.jpg')) {
```

Figuur 2 - Voorbeeld 2 van hardcoded images

```
// Controleren van de win-situaties
if (
    (
        // Rijen
        (
            // Rij 1
            element_cells[0].src.includes('img/cross200x200.png') &&
            element_cells[1].src.includes('img/cross200x200.png') &&
            element_cells[2].src.includes('img/cross200x200.png')
        ) || (
            // Rij 2
            element_cells[3].src.includes('img/cross200x200.png') &&
            element_cells[4].src.includes('img/cross200x200.png') &&
            element_cells[5].src.includes('img/cross200x200.png')
        ) || (
            // Rij 3
            element_cells[6].src.includes('img/cross200x200.png') &&
            element_cells[7].src.includes('img/cross200x200.png') &&
            element_cells[8].src.includes('img/cross200x200.png')
        )
    ) || (
        // Kolommen
        (
            // Kolom 1
            element_cells[0].src.includes('img/cross200x200.png') &&
            element_cells[3].src.includes('img/cross200x200.png') &&
```

Figuur 3 - Voorbeeld 3 van hardcoded images. Hier is het bijvoorbeeld zeer onduidelijk welke speler we nu bedoelen

Slecht te onderhouden code

Het voorbeeld bij 'Onleesbaarheid van de code' is ook gelijk een voorbeeld van slecht te onderhouden code.

Waarom? Nou stel we willen overstappen naar andere symbolen voor de spelers. Dus i.p.v. `cross200x200.png` voor speler 1 en `circle200x200.png` voor speler 2 willen dit veranderen naar `kruisje.png` voor speler 1 en `cirkel.png` voor speler 2.

Als we dit willen doen moeten we door de gehele code heen lopen op zoek naar de hardcoded vermeldingen van deze afbeeldingen.

Nou is het zo dat wat we gemaakt hebben absoluut geen groot project is, maar stel je voor dat je een dergelijke wijziging moet doorvoeren in een project wat bestaat uit 1000 bestanden met code. Je snapt dat dit niet meer te doen is dan, en dat de kans dat je het ergens overziet (vergeet) zeer groot is.

Wat gaan we doen?

We gaan het potentieel probleem (zie hierboven) aanpakken en voorkomen door:

- Onze code leesbaarder te maken
- Makkelijker te onderhouden in de toekomst.

REFACTORING SPELERNUMMERS

Inleiding

In onze code gebruiken we steeds gewoon een getal om een speler aan te duiden. Zo gebruiken we steeds het getal 1 voor speler 1 en het getal 2 voor speler 2. Maar als je deze getallen ziet word je in de code niet echt duidelijk dat het om een speler gaat en ook niet duidelijk welke relatie er is tussen b.v. een spelersnummer en de bijbehorende afbeelding (symbool).

Dit willen we op z'n minst leesbaarder maken in onze code.

Hoe pakken we dit aan?

We willen i.p.v. bijvoorbeeld het getal 1 in onze code, tenminste daar waar we een spelersnummer bedoelen, zoiets zien als `_PLAYER1`.

Dit is veel duidelijker dan een getal. Dit kunnen we doen door bovenaan in onze code, daar waar we de globale variabelen hebben aangemaakt, een nieuw soort variabele aan te maken. Namelijk een constante variabele.

We doen dit dan voor beide spelers. Dus `_PLAYER1` en `_PLAYER2`.

Waarom een constante variabele?

Een constante variabele krijgt gelijk bij het aanmaken van de variabele een waarde en deze waarde kan daarna in onze code niet meer veranderen. En in dit geval is het ook niet de bedoeling dat een spelersnummer verandert. Een spelersnummer is constant.

Bij constanten is het 'Best Practice' om de naam van een constante variabele in **HOOFDLETTERS** te schrijven en vooraf te laten gaan door een **underscore** (`_`).

Dus daarom `_PLAYER1`.

Hoe ziet dat er dan uit?

```
18 let element_cells;           // De cellen in het speelbord, hierin tonen we de afbeelding van een speler
19 let element_game_button;     // De knop om een ronde te starten of te resetten
20
21 // HELPER VARIABLES
22 let current_player = 1;       // Het nummer van de speler die aan de beurt is
23 let score_player1 = 0;        // Hier houden we de score van speler 1 bij
24 let score_player2 = 0;        // Hier houden we de score van speler 2 bij
25 let current_round = 0;        // Welke ronde is het
26 let timer_id;                 // ID van de interval, dit is de klok van een ronde
27 let elapsedTimeInSeconds;     // Verlopen ronde tijd in seconden
28
29 // CONSTANTEN
30 const _PLAYER1 = 1;           // Beter leesbaar in onze code dan het getal 1, is ook Index in _IMAGES
31 const _PLAYER2 = 2;           // Beter leesbaar in onze code dan het getal 2, is ook Index in _IMAGES
32
```

Figuur 4 - Constanten voor spelersnummers voor de leesbaarheid

Hoe ziet dan de rest van onze code eruit?

Als we nu overal in onze code het getal 1 (daar waar een spelersnummer bedoelen) vervangen door de constante **_PLAYER1** en het getal 2 vervangen door de constante **_PLAYER2** dan krijgen we bijvoorbeeld het volgende resultaat (we laten niet alle code zien, maar pikken er een paar voorbeeld uit):

VOORBEELD 1: VOOR DE WIJZIGING

```
element_turn_image.src = (current_player == 1 ? 'img/circle200x200.png' : 'img/cross200x200.png');
```

Figuur 5 - Voor de wijziging, onduidelijk is waar het getal 1 voor staat

VOORBEELD 1: NA DE WIJZIGING

```
element_turn_image.src = (current_player == _PLAYER1 ? 'img/circle200x200.png' : 'img/cross200x200.png');
```

Figuur 6 - Na de wijziging, nu is duidelijk dat we speler 1 bedoelen. Nu is zelfs enigszins duidelijk dat de eerste image van speler 1 is

VOORBEELD 2: VOOR DE WIJZIGING

```
// Wisselen van beurt  
current_player = (current_player == 1 ? 2 : 1);
```

Figuur 7 - Voor de wijziging. Ook hier is totaal niet duidelijk waar de getallen voor staan.

VOORBEELD 2: NA DE WIJZIGING

```
current_player = (current_player == _PLAYER1 ? _PLAYER2 : _PLAYER1);
```

Figuur 8 - Na de wijziging. Dit is toch veel leesbaarder?

REFACTORING HARDCODED IMAGES

Inleiding

We hebben op veel plaatsen in onze code de images hardcoded geprogrammeerd en daarmee maken we onze code niet flexibel en onderhoudbaar.

'Best Practice' is dat de namen van de afbeeldingsbestanden op een plaats staan in de vorm van een constante b.v. en dan op de plaatsen waar nodig kunnen hergebruiken. Wanneer we dan de afbeeldingen willen wijzigen en dus andere afbeeldingsbestanden willen gaan gebruiken is het veel werkbaarder en dus ook beter te onderhouden wanneer we dat op één plek in onze code kunnen doen.

Eén constante voor alle afbeeldingen

Afbeeldingsbestanden

In ons programma maken we gebruik van 3 afbeeldingsbestanden, namelijk:

1. *img/empty.jpg*
Om visueel een cel leeg te maken.
2. *img/cross200x200.png*
De afbeelding voor speler 1 in ons geval.
3. *img/circle200x200.png*
De afbeelding voor speler 2 in ons geval.

Namen van afbeeldingsbestanden in een variabele/constante

We willen bij voorkeur alle drie de namen van de afbeeldingsbestanden verzamelen in één constante.

Dit kan het beste met een eenvoudige **ARRAY**. En als we slim zijn dan plaatsen we de namen in een logische volgorde waarbij we rekening houden met de twee constanten die we al eerder hebben gemaakt, namelijk **_PLAYER1** en **_PLAYER2**.

Want hoe mooi en leesbaar zou het zijn als we één van deze twee constanten weer kunnen gebruiken als **index** voor de **ARRAY** die we gaan maken.

Dit betekent dat we de naam van het afbeeldingsbestand *img/empty.jpg* als eerste in de array moeten plaatsen, want deze krijgt dan de index waarde 0. Daarna plaatsen we de afbeelding van speler 1 in de array, zodat deze de index waarde 1 krijgt (zelfde waarde als van **_PLAYER1**) etc.....

De constante

```
const _IMAGES = [  
    'img/empty.jpg',           // Index 0, afbeelding lege Cel  
    'img/cross200x200.png',    // Index 1, afbeelding/symbool van speler 1  
    'img/circle200x200.png'    // Index 2, afbeelding/symbool van speler 2  
];
```

Figuur 9 - De constante voor images, een array

En wanneer we ergens in de code gebruik willen maken van de afbeelding voor speler 1 kunnen we de volgende manier gebruiken:

```
_IMAGES[_PLAYER1]
```

Figuur 10 - Verwijzen naar de image van speler 1

img/empty.jpg

Oeps

Oeps..... we hebben nu een mooi constante aangemaakt voor de afbeeldingsbestanden en twee voor de spelersnummers en we hebben het mogelijk gemaakt om van de constanten voor de spelersnummers ook gebruik te maken als index voor de array **_IMAGES**.

Maar als we gebruik willen maken in onze code van het afbeeldingsbestand *img/empty.jpg* ziet dat er nog steeds leesbaar en onderhoudbaar uit, want we zouden het dan als volgt gebruiken:

```
_IMAGES[0];
```

Figuur 11 - Verwijzen naar de 'lege' image. Nog steeds niet leesbaar.

Dit is nog steeds niet leesbaar en onderhoudbaar. Hoe kan ik immers aan de hand van dit stukje code zien dat we het afbeeldingsbestand *img/empty.jpg* bedoelen? Niet dus.

De oplossing

Dit kunnen we oplossen door nog een constante aan te maken hiervoor. En die noemen we dan **_EMPTY_CELL**.

Dus:

```
31  const _PLAYER2 = 2;           // Beter leesbaar in onze code dan het getal 2, is ook Index in _IMAGES
32
33  const _EMPTY_CELL = 0;       // Index van de array hieronder voor de afbeelding die een lege cel weergeeft
34
35  const _IMAGES = [
36    'img/empty.jpg',           // Index 0, afbeelding lege Cel
```

Figuur 12 - Constante voor een 'lege' image

Nu kunnen we voortaan de volgende code gebruiken wanneer we *img/empty.jpg* bedoelen:

```
_IMAGES[_EMPTY_CELL];
```

Figuur 13 - Verwijzen met de nieuwe constante naar een 'lege' image. Leesbaarder toch?

Dit is veel leesbaarder en onderhoudbaarder.

REFACTORING VAN CEL NUMMERS

Inleiding

In onze code hebben we heel veel van onderstaande blokken:

```
if (
    (
        // Rijen
        (
            // Rij 1
            element_cells[0].src.includes('img/cross200x200.png') &&
            element_cells[1].src.includes('img/cross200x200.png') &&
            element_cells[2].src.includes('img/cross200x200.png')
        ) || (
            // Rij 2
            element_cells[3].src.includes('img/cross200x200.png') &&
```

Figuur 14 - Cellen in het speelveld benaderen met indices

Hiermee hebben een complexe (gedrocht) **IF**-statement samengesteld om o.a. te controleren of een speler gewonnen heeft.

De cellen staan allemaal in de variabele `element_cells` en met een **index** waarde kunnen we iedere cel afzonderlijk benaderen in deze variabele. Zoals je ook hierboven kunt zien.

Als we coderen `element_cells[0]` bedoelen we de *eerste cel* van het speelveld. En let op: DE EERSTE CEL.

Als we coderen `element_cells[2]` bedoelen we de *derde cel* van het speelveld.

Je merkt dat we steeds letterlijk zeggen eerste cel, derde cel, etc.... En niet nulde cel, etc....

Zou het dan niet mooier zijn dat we constanten hebben die hoe we het uitspreken ook laat zien in de code?

Zoals `_CELL1` of `_CELL3`, etc....

Hoe ziet de code er dan uit?

```
41 // Leesbare constanten
42 const _CELL1 = 0;           // Indexwaarden voor de array met cellen
43 const _CELL2 = 1;           // Dit is ook leesbaarder in onze code dan
44 const _CELL3 = 2;           // gewoon getallen.
45 const _CELL4 = 3;
46 const _CELL5 = 4;
47 const _CELL6 = 5;
48 const _CELL7 = 6;
49 const _CELL8 = 7;
50 const _CELL9 = 8;
```

Figuur 15 - Constanten met leesbare namen voor cellen in het speelveld

Hoe ziet code er dan uit als we de constanten gaan gebruiken?

VOOR DE WIJZIGING

```
element_cells[0].src.includes(_IMAGES[_PLAYER1]) &&
element_cells[1].src.includes(_IMAGES[_PLAYER1]) &&
element_cells[2].src.includes(_IMAGES[_PLAYER1])
// Rij 1
```

Figuur 16 - Cellen benaderen met indices

NA DE WIJZIGING

```
// Rijen
element_cells[_CELL1].src.includes(_IMAGES[_PLAYER1]) && // Rij 1
element_cells[_CELL2].src.includes(_IMAGES[_PLAYER1]) &&
element_cells[_CELL3].src.includes(_IMAGES[_PLAYER1])
// Rij 2
```

Figuur 17 - Cellen benaderen met de leesbare constanten

REFACTORING IF-STATEMENT IN CELLCLICK FUNCTIE

Inleiding

De **IF**-statement zoals we die nu hebben geprogrammeerd en waarmee we o.a. controleren of een speler heeft gewonnen na iedere zet is complex, te groot en daarmee een gedrocht geworden.

Dit moet anders en kan ook anders. Maar let wel, de oplossing die wij nu gaan implementeren is ***niet de enige manier*** om het voor elkaar te krijgen. Er zijn altijd andere manieren, soms ook betere manieren zelfs. Maar om het niet te ingewikkeld te maken volstaan we met de oplossing zoals wij deze nu gaan implementeren.

Zoals het nu is

```
// Controleren van de win-situaties
if ( // SPELER 1 (cross)
    ( // Rijen
        ( // Rij 1
            element_cells[0].src.includes('img/cross200x200.png') &&
            element_cells[1].src.includes('img/cross200x200.png') &&
            element_cells[2].src.includes('img/cross200x200.png')
        ) || ( // Rij 2
            element_cells[3].src.includes('img/cross200x200.png') &&
            element_cells[4].src.includes('img/cross200x200.png') &&
            element_cells[5].src.includes('img/cross200x200.png')
        ) || ( // Rij 3
            element_cells[6].src.includes('img/cross200x200.png') &&
            element_cells[7].src.includes('img/cross200x200.png') &&
            element_cells[8].src.includes('img/cross200x200.png')
        )
    ) || ( // Kolommen
        ( // Kolom 1
            element_cells[0].src.includes('img/cross200x200.png') &&
            element_cells[3].src.includes('img/cross200x200.png') &&
            element_cells[6].src.includes('img/cross200x200.png')
        ) || ( // Kolom 2
            element_cells[1].src.includes('img/cross200x200.png') &&
            element_cells[4].src.includes('img/cross200x200.png') &&
            element_cells[7].src.includes('img/cross200x200.png')
        ) || ( // Kolom 3
            element_cells[2].src.includes('img/cross200x200.png') &&
            element_cells[5].src.includes('img/cross200x200.png') &&
            element_cells[8].src.includes('img/cross200x200.png')
        )
    ) || ( // Diagonalen
        ( // Diagonaal 1
            element_cells[0].src.includes('img/cross200x200.png') &&
            element_cells[4].src.includes('img/cross200x200.png') &&
            element_cells[7].src.includes('img/cross200x200.png')
        ) || ( // Diagonaal 2
            element_cells[2].src.includes('img/cross200x200.png') &&
            element_cells[4].src.includes('img/cross200x200.png') &&
            element_cells[6].src.includes('img/cross200x200.png')
        )
    )
) {
    // SPELER 1 HEEFT GEWONNEN
    // STAP 1 - Punten toekennen aan speler 1
    score_player1 += 2;
    // STAP 2 - Score tonen
    element_score_player1.innerHTML = score_player1;
    // STAP 3 - Ronde resetten
    element_game_button.click(); // TRUCJE :-}
} // EINDE SPELER 1 WIN CONTROLE
else if ( // SPELER 2 (circle)
    ( // Rijen
        ( // Rij 1
            element_cells[0].src.includes('img/cross200x200.png') &&
            element_cells[1].src.includes('img/cross200x200.png') &&
            element_cells[2].src.includes('img/cross200x200.png')
        ) || ( // Rij 2
            element_cells[3].src.includes('img/cross200x200.png') &&
            element_cells[4].src.includes('img/cross200x200.png') &&
            element_cells[5].src.includes('img/cross200x200.png')
        ) || ( // Rij 3
            element_cells[6].src.includes('img/cross200x200.png') &&
            element_cells[7].src.includes('img/cross200x200.png') &&
            element_cells[8].src.includes('img/cross200x200.png')
        )
    ) || ( // Kolommen
        ( // Kolom 1
            element_cells[0].src.includes('img/cross200x200.png') &&
            element_cells[3].src.includes('img/cross200x200.png') &&
            element_cells[6].src.includes('img/cross200x200.png')
        ) || ( // Kolom 2
            element_cells[1].src.includes('img/cross200x200.png') &&
            element_cells[4].src.includes('img/cross200x200.png') &&
            element_cells[7].src.includes('img/cross200x200.png')
        ) || ( // Kolom 3
            element_cells[2].src.includes('img/cross200x200.png') &&
            element_cells[5].src.includes('img/cross200x200.png') &&
            element_cells[8].src.includes('img/cross200x200.png')
        )
    ) || ( // Diagonalen
        ( // Diagonaal 1
            element_cells[0].src.includes('img/cross200x200.png') &&
            element_cells[4].src.includes('img/cross200x200.png') &&
            element_cells[7].src.includes('img/cross200x200.png')
        ) || ( // Diagonaal 2
            element_cells[2].src.includes('img/cross200x200.png') &&
            element_cells[4].src.includes('img/cross200x200.png') &&
            element_cells[6].src.includes('img/cross200x200.png')
        )
    )
) {
    // SPELER 2 HEEFT GEWONNEN
    // STAP 1 - Punten toekennen aan speler 2
    score_player2 += 2;
    // STAP 2 - Score tonen
    element_score_player2.innerHTML = score_player2;
    // STAP 3 - Ronde resetten
    element_game_button.click(); // TRUCJE :-}
} // EINDE SPELER 2 WIN CONTROLE
else if (
    isDraw() // Controleer op gelijkspel
)
```

Figuur 18 - Een gedrocht van een IF-statement

Helperfunction: Brainstormen

We zouden het volgende stuk code in een eigen functie kunnen plaatsen, waarbij we voor de functie een duidelijke beschrijvende naam geven die verteld hoe we de functie moeten gaan gebruiken en wat de functie doet:

```
element_cells[0].src.includes('img/cross200x200.png') &&  
element_cells[1].src.includes('img/cross200x200.png') &&  
element_cells[2].src.includes('img/cross200x200.png')
```

Figuur 19 - Geschikte code voor een eigen functie. Enige variabele waarde is een index van de array

Hoe maken we hier een eigen functie van?

Wat we niet moeten doen

De code, zoals we die nu hierboven zien, controleert voor speler 1 op een rij in het speelveld of de speler daar gewonnen heeft. Dit is op zich niet zo een-op-een over te zetten naar een eigen functie. Want dan hebben we er niet veel aan.

We zouden dan **9 functies (voor iedere rij, kolom en diagonaal) per speler** moeten maken. Dus in totaal **18 functies**, want ook voor speler 2 moeten we dit dan doen.

Dat maakt onze code niet beter, alleen de **IF**-statement wordt dan misschien leesbaarder.

Wat we wel moeten doen

We moeten de bovenstaande code aanpassen zodat we deze drie controles kunnen toepassen op iedere rij, kolom en diagonaal. Dit betekent dat we iedere hardcoded element moet vervangen door b.v. een variabele zodat bij gebruik van de functie kunnen aangeven:

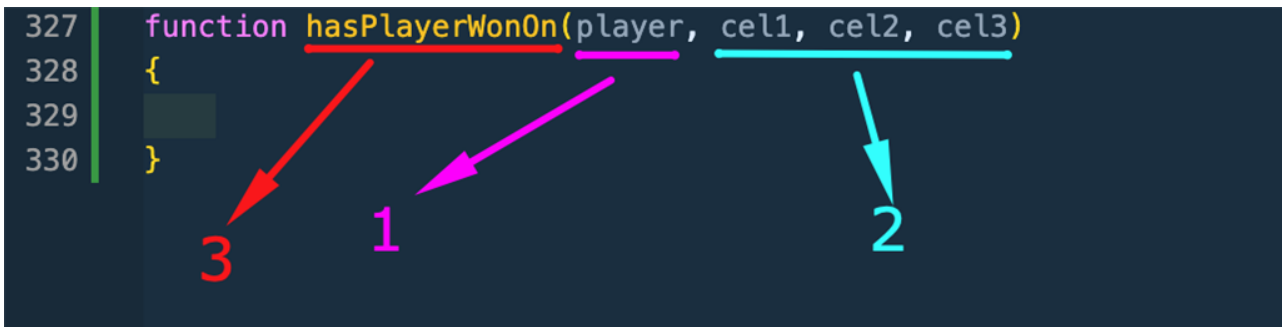
4. voor welke speler we controleren
5. in welke reeks van cellen we gaan controleren

Dit betekent dat we de functie, wanneer we deze gebruiken, informatie moeten geven m.b.t. de speler en de cellen.

Codering van de helperfunction

We declareren de functie als volgt:

```
327 function hasPlayerWonOn(player, cel1, cel2, cel3)
328 {
329     █
330 }
```



Figuur 20 - Helperfunction declaratie

Toelichting

1. Dit is de parameter waar het nummer van de speler plaatsen die gecontroleerd moet worden.
2. Dit zijn de parameters waar we de cel nummers plaatsen, daarmee maken we het mogelijk op rijen, op kolommen en op diagonalen te controleren.
3. De naam van de functie. De naam geeft zo duidelijk aan dat:
 - a. De functie controleert of een speler gewonnen heeft
 - b. Het deel van de naam **On** geeft aan dat we controleren op: *Rij* of *Kolom* of *Diagonaal*. Oftewel op een reeks cellen.
 - c. De naam geeft ook aan dat de waarde die het teruggeeft een **WAAR** of een **ONWAAR** oplevert.

Hoe gaan we de controle dan aanpassen?

CEL NUMMERS

VOOR DE AANPASSING

```
element_cells[0].src.includes('img/cross200x200.png') &&  
element_cells[1].src.includes('img/cross200x200.png') &&  
element_cells[2].src.includes('img/cross200x200.png')
```

Figuur 21 - Voor de aanpassing

Allereerst moeten we de cel nummers, zoals die hierboven hardcoded zijn, zo aanpassen dat we de **parameters cel1, cel2 en cel3** van de functie hiervoor gebruiken.

NA DE AANPASSING

```
element_cells[cel1].src.includes('img/cross200x200.png') &&  
element_cells[cel2].src.includes('img/cross200x200.png') &&  
element_cells[cel3].src.includes('img/cross200x200.png')
```

Figuur 22 - Na de aanpassing. Parameters cel1, cel2 en cel3 gebruiken

SPELERS AFBEELDING

Zoals je hierboven kunt zien is de afbeelding nu nog hardcoded en daarmee altijd gericht op speler 1. We kunnen ook hier weer gebruik maken van de constante `_IMAGES`, want daarin hebben we immers de namen van de afbeeldingen vastgelegd.

De parameter **player** van de functie bevat altijd een **1** (*speler 1*) of een **2** (*speler 2*). Als we dit weten dan weten we ook dat we de nummers die in deze parameter zitten kunnen gebruiken als **INDEX** voor de constante `_IMAGES`.

Als we deze wijziging doorvoeren dan krijgen we onderstaande aanpassing:

```
element_cells[cel1].src.includes(_IMAGES[player]) &&  
element_cells[cel2].src.includes(_IMAGES[player]) &&  
element_cells[cel3].src.includes(_IMAGES[player])
```

Figuur 23 - Hardcoded images vervangen

De functie een WAAR of ONWAAR terug laten geven

De functie moet altijd een **WAAR** of een **ONWAAR** teruggeven aan de code waar we de functie gebruiken. Om een functie een waarde terug te laten geven gebruiken we het keyword **return**.

Wat returnen we dan?

We returnen de uitkomst van de controle die we net hebben gemaakt:

```
return (  
    element_cells[cel1].src.includes(_IMAGES[player]) &&  
    element_cells[cel2].src.includes(_IMAGES[player]) &&  
    element_cells[cel3].src.includes(_IMAGES[player])  
)
```

Figuur 24 - Uitkomst van de controle gelijk met een return teruggeven

Let daarbij er wel op dat we de drie controles, want het zijn in feite drie, **tussen haakjes plaatsen**. We doen dit omdat we de uitkomst van alle drie de controles samen willen als een **WAAR** of een **ONWAAR**.

De complete helperfunction

Als we alles samenvoegen tot een complete functie en deze ook voorzien van commentaar dan krijgen we het volgende:

```
308  /**
309   * hasPlayerWonOn
310   * -----
311   * Met deze functie kunnen we controleren of een speler in meegegeven 3 cellen
312   * zijn/haar symbool heeft staan, want dan heeft de speler gewonnen en geeft deze
313   * functie een WAAR terug.
314   *
315   * PARAMETERS:
316   * -----
317   * player      getal      1 = Speler 1, 2 = Speler 2
318   * cell1       getal      Nummer van de eerste cel
319   * cel2       getal      Nummer van de tweede cel
320   * cel3       getal      Nummer van de derde cel
321   *
322   * RETURN:
323   * -----
324   * WAAR        Als het symbool van de speler in de 3 gegeven cellen staat
325   * ONWAAR      Als het symbool van de speler niet in alle 3 de gegeven cellen staat
326   */
327  function hasPlayerWonOn(player, cel1, cel2, cel3)
328  {
329      return (
330          element_cells[cel1].src.includes(_IMAGES[player]) &&
331          element_cells[cel2].src.includes(_IMAGES[player]) &&
332          element_cells[cel3].src.includes(_IMAGES[player])
333      )
334  }
```

Figuur 25 - De complete helperfunction

DE NIEUWE HELPERFUNCTION GEBRUIKEN

Inleiding

We gaan de nieuwe helperfunction nu in de complexe IF-statement toepassen en we zullen zien dat dit er al een stuk beter en leesbaarder uit gaat zien.

De nieuwe IF-statement

```
202 // Controleren van de win-situaties
203 if ( // SPELER 1 (cross)
204     ( // Rijen
205         hasPlayerWonOn(_PLAYER1, _CELL1, _CELL2, _CELL3) || // Rij 1
206         hasPlayerWonOn(_PLAYER1, _CELL4, _CELL5, _CELL6) || // Rij 2
207         hasPlayerWonOn(_PLAYER1, _CELL7, _CELL8, _CELL9) // Rij 3
208     ) || ( // Kolommen
209         hasPlayerWonOn(_PLAYER1, _CELL1, _CELL4, _CELL7) || // Kolom 1
210         hasPlayerWonOn(_PLAYER1, _CELL2, _CELL5, _CELL8) || // Kolom 2
211         hasPlayerWonOn(_PLAYER1, _CELL3, _CELL6, _CELL9) // Kolom 3
212     ) || ( // Diagonalen
213         hasPlayerWonOn(_PLAYER1, _CELL1, _CELL5, _CELL9) || // Diagonaal 1
214         hasPlayerWonOn(_PLAYER1, _CELL3, _CELL5, _CELL7) // Diagonaal 2
215     )
216 ) {
217     // SPELER 1 HEEFT GEWONNEN
218     // STAP 1 - Punten toekennen aan speler 1
219     score_player1 += 2;
220     // STAP 2 - Score tonen
221     element_score_player1.innerHTML = score_player1;
222     // STAP 3 - Ronde resetten
223     element_game_button.click(); // TRUCJE :-)
224
225 } // EINDE SPELER 1 WIN CONTROLE
226 else if ( // SPELER 2 (circle)
227     ( // Rijen
228         hasPlayerWonOn(_PLAYER2, _CELL1, _CELL2, _CELL3) || // Rij 1
229         hasPlayerWonOn(_PLAYER2, _CELL4, _CELL5, _CELL6) || // Rij 2
230         hasPlayerWonOn(_PLAYER2, _CELL7, _CELL8, _CELL9) // Rij 3
231     ) || ( // Kolommen
232         hasPlayerWonOn(_PLAYER2, _CELL1, _CELL4, _CELL7) || // Kolom 1
233         hasPlayerWonOn(_PLAYER2, _CELL2, _CELL5, _CELL8) || // Kolom 2
234         hasPlayerWonOn(_PLAYER2, _CELL3, _CELL6, _CELL9) // Kolom 3
235     ) || ( // Diagonalen
236         hasPlayerWonOn(_PLAYER2, _CELL1, _CELL5, _CELL9) || // Diagonaal 1
237         hasPlayerWonOn(_PLAYER2, _CELL3, _CELL5, _CELL7) // Diagonaal 2
238     )
239 ) {
240     // SPELER 2 HEEFT GEWONNEN
241     // STAP 1 - Punten toekennen aan speler 2
242     score_player2 += 2;
243     // STAP 2 - Score tonen
244     element_score_player2.innerHTML = score_player2;
245     // STAP 3 - Ronde resetten
246     element_game_button.click(); // TRUCJE :-)
247
248 } // EINDE SPELER 2 WIN CONTROLE
249 else if (
250     isDraw() // Controleer op gelijkspel
251 ) {
```

Figuur 26 - Nieuwe IF-statement door gebruik te maken van de helperfunctie

Vergelijk dit eens met zoals het eerst was.